

Mid-Term Report: SortNow

1. Task Description

My project goal is to build a waste object detection system that can identify and classify waste into six categories: biodegradable, cardboard, glass, metal, paper, and plastic . This requires both identifying the waste type and localizing it with bounding boxes, handling image variations like lighting, orientations, and cluttered scenes.

For this I initially explored YOLOv8 [\[1\]](#) to understand multiple-object detection, as it was used in many research papers having similar goals. I then trained my dataset on YOLOv11 because it had a higher accuracy and faster detection speed on test images.

However, to gain a deeper understanding of computer vision fundamentals, and after feedback from my mentor, I decided to build a custom object detection CNN from scratch. I started with making a single-object detector [\[2\]](#), defining my own layers and architecture. This helped me understand the inner workings and core concepts beyond just using pre-trained models.

Key insights I gained was the trade-off between transfer learning and training from scratch, and how architectural choices matter because deeper networks learn more features but are prone to overfitting on small datasets (which happened in my case). Also, how data quality and preprocessing are as critical as model architecture.

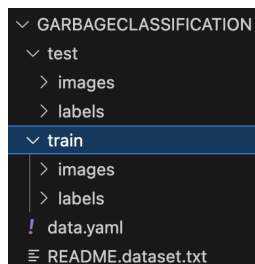
Based on my learning, I am now implementing a YOLO v1 like multiple-object detection model with a simpler architecture [\[3\]](#) .

2. Dataset [\[4\]](#):

Obtained this dataset online from roboflow, which I downloaded and modified locally on my laptop. It had pre-marked bounding boxes, which I downloaded in YOLO format.

Each image is a jpg file and has a corresponding txt file with the specific object class, and bounding box coordinates as x center, y center, width, height of box in separate lines for each object in an image.

It was pre-organized into train, validate, test folders, and has a data.yaml file with names of classes corresponding to label indexes. I checked the dataset for duplicates and corrupt images (ones with no bounding box)



For the single-object detection model:
MODIFICATIONS:

- I deleted all images and corresponding labels with multiple objects per image, checked for duplicates and corrupt images.
- I shifted all files in the validate folder to test folder, as the dataset was small and used only test- train split to train and evaluate my model.

RESULTING DATASET:

- Number of classes=6

	BIODEGRADABLE	CARDBOARD	GLASS	METAL	PAPER	PLASTIC
images in test dir	143	268	380	205	374	128
images in train dir	348	626	897	514	907	391

- Total number of images in train: 3,583
- Total number of images in test: 1,498

For the multiple- object detection model:

MODIFICATIONS:

- I deleted all images and corresponding labels with incorrect and over-labelling(labels>98) and I shifted all files in the validate folder to test folder

RESULTING DATASET:

	BIODEGRADABLE	CARDBOARD	GLASS	METAL	PAPER	PLASTIC
images in test dir	646	439	794	374	513	348
images in train dir	1491	1016	1851	873	1215	814

- Total number of images in train: 7,260
- Total number of images in test: 3,114

NOTE: The single-object dataset (3,583 train/1,498 test) was used for the custom CNN model. The multi-object dataset (7269 train/3114 test) will be used for YOLO v1 training.

CHALLENGES: not being able to find a properly labeled dataset with single object per image and sufficient number of images. Overcame it by using a smaller dataset but applying augmentations during training (random horizontal flip, random rotation, resized crop, color jitter)

3. Model Architecture

Single-Object Detection Model:

Input: Single waste item images in JPG format (resized to 448x448)

Processing:

- Image converted to tensor
- 3 convolutional blocks (64, 128, 256 filters) and in each block: Conv2d, BatchNorm, ReLU, MaxPool
- Flatten layer that results in 2 output heads: for classification and bounding box coordinates
- Classification and bounding box prediction

Output: Waste class label and 4 bounding box coordinates, which was then marked on the image using OpenCV

Understanding: Through experimentation, I learned fewer layers (3 vs deeper) reduced overfitting on my limited dataset.

YOLO inspired Multi-Object Model Architecture:

Input: Images of mixed/ single waste items in JPG format

Processing Stages:

- This input image is resized to 448x448 and normalized, then converted to tensor using the torchvision library.
- Feature extraction is done by passing through the convolutional layers (Architecture code implemented by following reference tutorials, but design not final yet)
- It divides the image into a grid, and each grid cell predicts bounding boxes and class probabilities for objects in their region. This allows simultaneous predictions for multiple objects.
- bounding box regression and classification both happen in the same network. bounding box coordinates are detected and classification of waste type is done in a single forward pass
- Post processing calculates confidence value.

Output: Predicted waste class labels and bounding box coordinates for detected objects

4. Implementation

Completed Work:

- Explored YOLOv8 architecture and documentation
- Completed tutorials on flask, python, numpy, matplotlib, basics of pytorch
- Successfully trained YOLOv11 models on the modified dataset using transfer learning.
- Implemented and trained a custom single-object detection model from scratch using CNN with dataset preprocessing (duplicate removal, corrupt image removal, and single-object filtering)
- Applied data augmentation techniques in code to compensate for limited dataset size

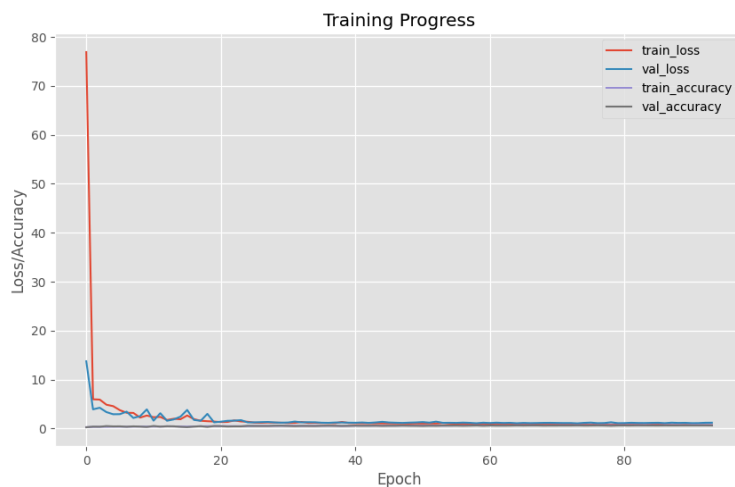
Results and Observations:

- The custom single-object detection model achieved 66.09% validation accuracy and validation loss of 1.070 at epoch 74.
- Training was completed in 41.51 minutes (after 94 epochs because early stopping was triggered).
- The model showed signs of overfitting, as final training accuracy was 74.30% while validation accuracy decreased to 63.28%.
- However, when tested on new, unseen waste images, it demonstrated better performance than expected, correctly detecting and classifying unseen waste objects.

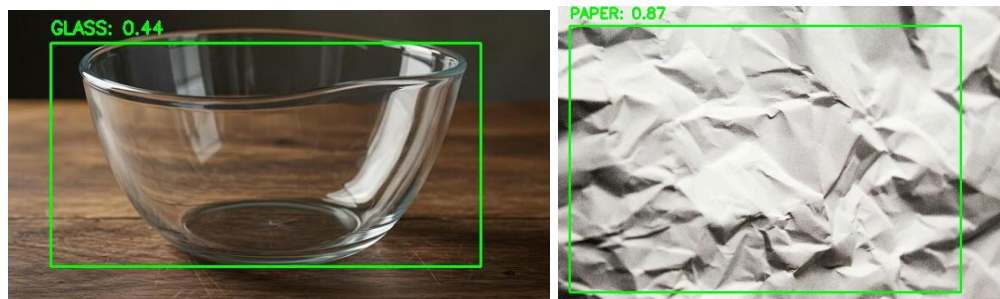
Current Status:

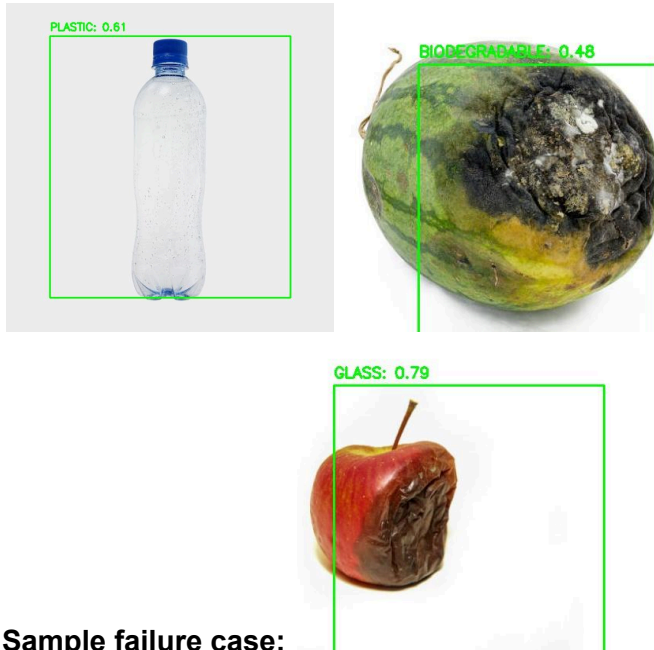
- Implemented YOLO v1 code for dataset reading and loss by following reference code from tutorials.
- Currently understanding core components, including the grid mechanism, loss function in depth
- exact architecture undecided and training not yet started
- Flask integration not yet started
- Frontend using HTML and CSS made.

Single-Object Detection Model test results and sample images



Sample success cases:





Sample failure case:

5. End-Term Goals with timeline

Dec 22-28th:

High priority:

- Complete model architecture and train the YOLO v1-inspired multi-object detection model with the full dataset
- Evaluate model performance on multiple objects per image scenarios

Medium priority:

- Develop Flask backend integration to connect the trained model with the frontend interface
- Enable local deployment on my laptop for real-time image input and waste classification predictions

Stretch Goals (If Time Permits):

- Deploy the application to a cloud platform (Render or similar)
-

6. References

1. [research paper for official yolo architecture](#)
2. [cnn in pytorch](#)
3. [yolov1 from scratch](#)
4. [roboflow dataset for project](#)

5. [writing cnn from scratch using pytorch](#)
 6. [cnn from scratch, for troubleshooting](#)
 7. [github for yolov1 architecture project](#)
 8. [geeksforgeeks, to learn all ML concepts in depth](#)
 9. [pytorch tutorials](#)
 10. [pillow PIL and image handling in python](#)
 11. [to understand yolov8](#)
 12. [for yolo models results evaluation](#)
-

Additional Notes

- **Project Scope Change:** After discussing with my mentor, I shifted focus from only using pre-trained models to building models from scratch. Also , I changed from cloud deployment to local deployment on my laptop.
 - **Vacation Schedule:** Will be traveling to Australia from 21st Dec - Jan 7th, so end-term goals were adjusted accordingly in consultation with mentor
-